Course Outline: Git for Developers

Title: Git for Developers: Version Control Fundamentals **Prerequisite:** Command Line Basics (Week 2, Day 4) **Target Audience:** Beginner developers learning version control for the first time **Total Lessons:** 22 **Estimated Total Length:** ~11,000 words **Format:** Practical (45% hands-on)

Course Description

This course introduces students to Git, the industry-standard version control system. Students will learn how Git tracks changes, manages project history, and enables collaboration through branching and merging. The course emphasizes practical workflows that prepare students for real development environments, including how to work with Git through the command line, IDE interfaces, and AI-assisted tools.

By the end of this course, students will confidently navigate Git repositories, create and merge branches, manage their project history, and follow professional Git workflows. This foundation prepares them for collaborative development covered in the subsequent GitHub course.

Course Philosophy

This course emphasizes:

- **Understanding over memorization**: Learn how Git thinks, not just commands
- **Progressive complexity**: Start with essential commands, add advanced techniques gradually
- **Multiple interfaces**: Experience Git through CLI, IDE, and AI assistance
- **Real-world workflows**: Practice patterns used in professional development
- **Path of least resistance**: Learn the simplest approach first, understand alternatives later

---

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Git Fundamentals	4	~2,200
02 - Working with Branches	4	~2,300
03 - Advanced Git Operations	4	~2,300
04 - Git Workflows	4	~2,200
05 - Assessment	2	~1,200
06 - Conclusion	1	~450
Total	**22**	**~11,000**

Section 00: Welcome to Git

00.0 Welcome: Version Control for Developers 📖 (~450 words)

Learning Objectives:

- Understand what version control is and why developers need it
- Recognize how Git solves real development problems
- Connect Git to collaborative AI-powered development

Content Outline:

- The problem: Working on code without losing history
- What Git provides: Time travel for your code
- Why Git matters: Industry standard, collaboration enabler
- How Git fits into the 4Geeks philosophy: Enabling confident iteration
- What you'll learn in this course
- Connection to AI-assisted development

Transitions: This welcome establishes why Git matters before diving into how it works in Section 01.

Key Principles:

- Version control enables confident experimentation
- Git is a skill that compounds with other development abilities
- Understanding Git's mental model matters more than memorizing commands
- Git works alongside AI tools, not against them

Examples:

- Scenario: Making a change that breaks your code—how do you get back?
- Real situation: Working on a feature while your teammate works on another
- Common mistake: Losing work because you didn't track changes

---

Section 01: Git Fundamentals

01.0 Understanding Git: How Version Control Works 📖 (~550 words)

Learning Objectives:

- Understand Git's snapshot-based tracking system
- Recognize the three states: working directory, staging area, committed
- Visualize how Git stores project history
- Connect commits to project features or milestones

Content Outline:

- What Git actually is: A time machine for your code
- The three-state system: Working, staging, committed
- Snapshots vs. file differences: How Git thinks
- The commit: Your project's checkpoint
- Repository structure: The .git folder
- Local vs. remote: Two copies of your history

Transitions: Understanding how Git works mentally prepares you for the commands you'll learn in the next lesson. The three-state system explains why we need both `git add` and `git commit`.

Key Principles:

- Git tracks snapshots of your entire project, not individual file changes
- Each commit represents a complete, working state of your project
- The staging area lets you craft intentional commits
- Commits should relate to specific features or changes

Examples:

- Visual: The three states with file icons
- Analogy: Commits as save points in a video game
- Real scenario: Making multiple changes but committing them separately
- Counter-example: A messy commit that changes 20 unrelated things

---

01.1 Your First Git Commands: Status, Add, and Commit 💻 (~600 words)

Learning Objectives:

- Initialize a Git repository
- Check repository status with `git status`
- Stage changes with `git add`
- Create commits with `git commit`

Content Outline:

- Setting up Git: `git config` basics (name and email)
- Creating a repository: `git init`
- Checking what's changed: `git status` (your most-used command)
- Staging specific files: `git add <filename>`
- Creating a commit: `git commit -m "message"`
- Writing good commit messages: Clear, descriptive, action-oriented

Transitions: Now that you understand Git's mental model, these commands will make sense. Each command corresponds to moving files through the three states you learned about.

Key Principles:

- Always check status before and after operations
- Stage intentionally—don't just `git add .` everything
- Commit messages should explain *why*, not just *what*
- One logical change per commit

Examples:

- Creating your first repository
- Making a change and seeing it with `git status`
- Staging and committing a new file
- Checking status after commit (clean working directory)

Hands-On Exercise: Create a simple project directory with an HTML file. Initialize Git, make changes to the file, check status, stage changes, and create your first commit. Practice the full cycle multiple times with different changes.

Success Criteria:

- Repository initialized successfully
- Changes staged and committed
- Commit messages are clear and descriptive
- Student can explain what each command does

01.2 The HEAD Concept: Where Are You in Git? 📖 (~500 words)

Learning Objectives:

- Understand what HEAD represents in Git
- Visualize your current position in project history

- Recognize how HEAD moves with commits and checkouts
- Connect HEAD to navigation concepts

Content Outline:

- What is HEAD? Your current position in the project timeline
- HEAD as a pointer: Always pointing to your current location
- Detached HEAD state: What it means and why it happens
- HEAD in relation to branches (preview for next section)
- Moving through history: Viewing different commits
- Why knowing where you are matters

Transitions: Understanding HEAD is crucial before learning about branches and remote repositories. This concept explains where you are in your project's timeline and prepares you for more complex Git operations.

Key Principles:

- HEAD tells you which commit you're currently viewing
- Most operations happen relative to HEAD
- Knowing where you are prevents confusion and mistakes
- HEAD is fundamental to understanding Git navigation

Examples:

- Visualization: HEAD as a "you are here" marker on a timeline
- Diagram: HEAD pointing to different commits
- Real scenario: Checking out an old commit to see previous work
- Common confusion: "Where did my changes go?" (HEAD moved)

01.3 Working with Remote Repositories 💻 (~550 words)

Learning Objectives:

- Understand local vs. remote repositories
- Connect a local repository to a remote with `git remote`
- Push commits to a remote with `git push`
- Pull changes from a remote with `git pull`

Content Outline:

- Local vs. remote: Two copies, synchronized
- Why remotes matter: Backup, collaboration, deployment
- Adding a remote: `git remote add origin <url>`
- Checking remotes: `git remote -v`
- Pushing changes: `git push origin main`
- Pulling changes: `git pull origin main`
- The origin convention: Standard naming for primary remote

Transitions: You've been working locally; now it's time to connect to remote repositories. This enables backup and sets the foundation for collaboration (which you'll explore with GitHub in the next course).

Key Principles:

- Remote repositories enable collaboration and backup
- Push sends your commits to the remote
- Pull brings remote commits to your local repository
- Always pull before pushing to avoid conflicts

Examples:

- Setting up a remote repository (generic remote, not GitHub-specific)
- Pushing your first commits
- Simulating a change on the remote and pulling it
- Understanding the relationship between local and remote

Hands-On Exercise: Create a remote repository (provide generic instructions). Connect your local repository to the remote, push your commits, make a change on the remote, and pull those changes back to local. Experience the full local-remote cycle.

Success Criteria:

- Remote added successfully
- Commits pushed to remote
- Changes pulled from remote
- Student understands the local-remote relationship

Section 02: Working with Branches

02.0 Understanding Branches: Parallel Development 📖 (~550 words)

Learning Objectives:

- Understand what branches are and why they exist
- Visualize parallel development with multiple branches
- Recognize when to create a new branch
- Connect branches to feature development workflow

Content Outline:

- What is a branch? A parallel version of your project
- The main branch: Your stable production code
- Why branches matter: Safe experimentation and feature isolation
- Branches as cheap copies: Git's lightweight approach
- Branch naming conventions: feature/, bugfix/, dev
- How branches relate to commits and HEAD
- The branch workflow: Create, work, merge

Transitions: You've mastered linear Git workflow (commit after commit on one branch). Now learn how to work on multiple features simultaneously through branching—a core skill for professional development.

Key Principles:

- Branches enable working on multiple features without interference
- Each branch is an independent line of development
- Main branch should always contain stable, working code
- One feature = one branch (clear separation of concerns)

Examples:

- Visualization: Timeline with multiple branches
- Real scenario: Working on a new feature while main branch stays stable
- Team scenario: Multiple developers, each on their own branch
- Analogy: Branches as parallel universes of your project

02.1 Creating and Managing Branches 💻 (~600 words)

Learning Objectives:

- Create new branches with `git branch`
- Switch between branches with `git switch`
- List all branches with `git branch -a`
- Delete branches after they're merged

Content Outline:

- Viewing current branches: `git branch`
- Creating a new branch: `git branch feature/new-feature`
- Switching branches: `git switch branch-name`
- Creating and switching in one command: `git switch -c branch-name`
- Understanding what switches when you change branches
- Working on a branch: Normal commit workflow
- Listing all branches including remotes: `git branch -a`
- Deleting branches: `git branch -d branch-name`

Transitions: Understanding branches conceptually prepares you for creating and navigating them. These commands give you the tools to implement the parallel development workflow.

Key Principles:

- Always know which branch you're on (check before committing)
- Create descriptive branch names (feature/user-login, bugfix/header-spacing)
- Clean up merged branches to keep repository tidy
- Commit your work before switching branches

Examples:

- Creating a feature branch for a new component
- Switching between main and feature branch
- Seeing different file states on different branches
- Deleting a branch after merging

Hands-On Exercise: Starting from main branch, create a new feature branch, make changes and commit them, switch back to main (see changes disappear), switch back to feature branch (see changes return), and create another branch to practice navigation. Experience how Git maintains different states across branches.

Success Criteria:

- Multiple branches created
- Successfully switched between branches
- Changes isolated to specific branches
- Student comfortable with branch navigation

02.2 Switch vs Checkout: Modern Git Navigation 📖 (~500 words)

Learning Objectives:

- Understand the difference between `git switch` and `git checkout`
- Recognize why Git introduced `git switch`
- Know when to use each command
- Apply modern Git best practices

Content Outline:

- The history: `git checkout` did too many things
- Modern approach: Separate commands for separate tasks
- `git switch`: For changing branches

- `git checkout` : For restoring files (less common now)
- The `-b` flag: Creating and switching simultaneously
- Why modern Git split checkout's functionality
- Best practice: Use `git switch` for branches

Transitions: You've been using `git switch` to navigate branches. This lesson explains why it exists and how it differs from older `git checkout` commands you might see in tutorials or AI-generated code.

Key Principles:

- Modern Git provides clearer, more focused commands
- `git switch` is the preferred way to change branches
- Understanding both commands helps you read existing code and tutorials
- Clearer commands lead to fewer mistakes

Examples:

- Comparison: `git switch feature` vs `git checkout feature`
- Creating and switching: `git switch -c feature` vs `git checkout -b feature`
- When you might still see checkout (older tutorials, legacy scripts)
- Reading error messages from both commands

02.3 Your First Branch Workflow 💻 (~650 words)

Learning Objectives:

- Execute a complete feature branch workflow
- Practice switching between main and feature branches
- Experience the safety of branch isolation
- Apply professional branching patterns

Content Outline:

- Starting point: Clean main branch
- Step 1: Create feature branch
- Step 2: Develop on feature branch (multiple commits)
- Step 3: Switch back to main (verify stability)
- Step 4: Preview merging (next section covers details)
- Why this workflow matters: Safety and organization
- Naming your branches: feature/, bugfix/ prefixes
- Managing multiple active branches

Transitions: You've learned branch commands individually; now combine them into the workflow you'll use for every feature you build. This workflow becomes second nature with practice.

Key Principles:

- Start every feature on a new branch
- Main branch remains stable while you experiment
- Descriptive branch names document what you're working on
- Complete features before creating new branches

Examples:

- Complete workflow: Creating a login feature on a branch
- Parallel development: Starting another feature while first is incomplete
- Safety demonstration: Breaking code on feature branch doesn't affect main
- Team simulation: Multiple features in development simultaneously

Hands-On Exercise: Create a simple multi-file project on main branch. Create a feature branch and add a new feature (new file or significant changes). Make multiple commits on the feature branch. Switch back to main and verify your feature work doesn't appear there. Create a second feature branch from main and work on something different. Practice managing multiple branches.

Success Criteria:

- Multiple feature branches created from main
- Work isolated on appropriate branches
- Clean commits on each branch
- Student can explain the workflow and its benefits

Section 03: Advanced Git Operations

03.0 Git Log: Reading Your Project History 📖 (~550 words)

Learning Objectives:

- View commit history with `git log`
- Interpret log output and commit information
- Use log options to get specific information
- Navigate project history effectively

Content Outline:

- Basic log: `git log` (viewing all commits)
- Understanding log output: Hash, author, date, message
- Condensed view: `git log --oneline`
- Detailed view: `git log --stat` (see what changed)
- Filtering logs: By author, date, or file
- Visualizing branches: `git log --graph`
- Finding specific commits: When something changed
- Why log matters: Understanding project evolution

Transitions: You've been creating commits; now learn to read the history you've built. Understanding your project's timeline is essential before learning to modify it with revert, reset, and stash.

Key Principles:

- Git log is your project's diary
- Commit hashes uniquely identify specific snapshots
- Good commit messages make log valuable
- Understanding history helps you make better decisions

Examples:

- Reading a basic log output
- Using --stat to see what files changed
- Finding when a bug was introduced by reading commit messages
- Visualizing branch history with --graph

03.1 Managing Changes: Stash, Revert, and Reset 💻 (~650 words)

Learning Objectives:

- Temporarily save work with `git stash`
- Undo commits safely with `git revert`
- Reset to previous states with `git reset`

- Choose the appropriate command for each situation

Content Outline:

- The stash: Temporary storage for uncommitted changes
- When to stash: Switching branches with uncommitted work
- Stashing: `git stash save "message"`
- Retrieving stashed changes: `git stash pop`
- Undoing commits: `git revert <commit-hash>`
- How revert works: Creating a new "opposite" commit
- Reset basics: `git reset --soft`, `--mixed`, `--hard`
- When to use each: Stash vs revert vs reset
- Danger zones: When reset deletes work

Transitions: Git log showed you history; now learn to modify it. These commands let you undo mistakes, temporarily save work, and manage your project state—essential skills for confident development.

Key Principles:

- Stash for temporary work (switching context)
- Revert for undoing pushed commits (safe for collaboration)
- Reset for local-only changes (use carefully)
- Always commit or stash before switching branches

Examples:

- Stashing work to switch branches urgently
- Reverting a commit that introduced a bug
- Resetting to undo local commits not yet pushed
- Recovering from an accidental reset (if possible)

Hands-On Exercise: Make changes without committing, then practice stashing them. Switch branches, then pop the stash. Create several commits and practice reverting one. Practice different reset modes (--soft, --mixed) on unpushed commits. Experience each command's effect on your working directory, staging area, and commit history.

Success Criteria:

- Successfully stashed and retrieved work
- Reverted a commit correctly
- Used reset appropriately (without losing work)
- Student can explain when to use each command

03.2 Merge vs Rebase: Two Ways to Integrate Changes 📖 (~550 words)

Learning Objectives:

- Understand how merge combines branches
- Understand how rebase rewrites history
- Recognize when to use each approach
- Apply simplified gitflow best practices

Content Outline:

- The integration challenge: Bringing feature work back to main
- Merge: Combining histories with a merge commit
- How merge works: Three-way merge
- Rebase: Replaying commits on a new base
- How rebase works: Rewriting commit history
- Merge vs rebase: Trade-offs

- When to merge: Finishing features, keeping complete history
- When to rebase: Cleaning up before merging (advanced)
- Simplified workflow: Merge for beginners

Transitions: You've worked on feature branches; now learn how to bring that work back to main. Understanding merge and rebase helps you choose the right integration strategy and understand what your AI tools are doing.

Key Principles:

- Merge preserves complete history (safer for beginners)
- Rebase creates cleaner history but rewrites commits
- Never rebase commits you've already pushed (breaks collaboration)
- For beginners: Use merge, understand rebase exists

Examples:

- Visualization: Merge creates a join point in history
- Visualization: Rebase moves commits to new base
- Real scenario: Finishing a feature and merging to main
- Warning example: Rebase gone wrong (team confusion)

03.3 Practicing Advanced Git Operations 💻 (~550 words)

Learning Objectives:

- Execute a complete merge workflow
- Practice resolving simple merge conflicts
- Apply log, stash, and merge together
- Build confidence with advanced operations

Content Outline:

- Scenario: Completing and merging a feature branch
- Step 1: Review your feature work with log
- Step 2: Ensure main is up to date
- Step 3: Merge feature into main: `git merge feature-branch`
- Understanding merge conflicts: Why they happen
- Resolving conflicts: Choosing which changes to keep
- After merge: Deleting the feature branch
- Practice workflow: Create, develop, merge, clean up

Transitions: You've learned advanced operations individually; now combine them in realistic scenarios. This practice builds the muscle memory and confidence needed for real development work.

Key Principles:

- Always check where you are before merging
- Review your feature work before merging
- Clean up branches after successful merges
- Conflicts are normal and solvable

Examples:

- Complete merge workflow with no conflicts
- Merge with simple conflict (both branches changed same line)
- Using log to verify merge success
- Cleaning up after merge

Hands-On Exercise: Create a feature branch and develop a small feature with multiple commits. While on feature branch, switch to main and make a different change (to simulate parallel work). Merge feature into main. Practice resolving a simple conflict if it occurs. Use log to see the merge commit. Delete the feature branch after successful merge. Repeat the process with a new feature.

Success Criteria:

- Successfully merged feature branch to main
- Resolved any conflicts correctly
- Verified merge with git log
- Cleaned up merged branches
- Student comfortable with full workflow

Section 04: Git Workflows and Best Practices

04.0 Simplified Gitflow: Main, Dev, Feature, and Bugfix 📖 (~550 words)

Learning Objectives:

- Understand the simplified gitflow branching model
- Recognize the purpose of each branch type
- Apply consistent naming conventions
- Implement professional Git workflows

Content Outline:

- What is gitflow? A branching strategy for organized development
- Main branch: Production-ready, stable code
- Dev branch: Integration branch for features
- Feature branches: feature/descriptive-name
- Bugfix branches: bugfix/issue-description
- The workflow: feature → dev → main
- Why structure matters: Clarity and safety
- Naming conventions: Consistency across teams
- Avoiding personal branch names (smith-branch, maria-test)

Transitions: You've learned Git mechanics; now learn the organizational patterns professionals use. This structure prevents chaos in team environments and establishes good habits early.

Key Principles:

- Consistent naming enables automation and understanding
- Clear branch purposes prevent mistakes
- Main branch is sacred (always stable)
- Feature branches enable parallel development
- Standard workflows enable better collaboration

Examples:

- Branch structure diagram: main, dev, and multiple features
- Good naming: feature/user-authentication, bugfix/header-alignment
- Bad naming: johns-stuff, temp, test, new-branch-2
- Real workflow: Developing a feature through the full cycle

04.1 Change Management: CLI vs IDE vs AI-Assisted 💻 (~600 words)

Learning Objectives:

- Execute Git commands via command line
- Use IDE Git interfaces effectively
- Leverage AI tools for Git operations
- Choose the appropriate interface for each task

Content Outline:

- Three ways to work with Git: Each has strengths
- Command line: Direct, precise, scriptable
- IDE interface: Visual, integrated, convenient
- AI assistance: Generating commands, explaining operations
- When to use CLI: Learning, precision, scripting
- When to use IDE: Quick operations, visual diffs
- When to use AI: Complex commands, explanations
- Best practice: Understand CLI even if you use IDE
- Reading AI-generated Git commands critically

Transitions: You've been using Git commands; now see how the same operations work through different interfaces. Understanding all three approaches makes you more effective and helps you collaborate with diverse teams.

Key Principles:

- CLI understanding enables using any interface
- IDE tools speed up common operations
- AI assistance helps but requires verification
- Know what's happening regardless of interface
- Command line skills are portable across all environments

Examples:

- Same operation (commit) via CLI, IDE, and AI
- When CLI is faster (bulk operations)
- When IDE is better (resolving merge conflicts visually)
- When AI helps (complex rebase commands)

Hands-On Exercise: Perform the same Git workflow using all three approaches: Make changes and commit via CLI, then repeat the process using your IDE's Git interface, and finally use AI to generate and execute Git commands. Compare the experiences and understand when each approach is most efficient.

Success Criteria:

- Completed Git operations via CLI
- Successfully used IDE Git interface
- Worked with AI to execute Git commands
- Student can articulate benefits of each approach

04.2 Writing Quality Commits: Git Best Practices 📖 (~500 words)

Learning Objectives:

- Write clear, descriptive commit messages
- Stage files intentionally for focused commits
- Organize commits logically around features
- Apply professional commit standards

Content Outline:

- What makes a good commit message
- Structure: Short summary, optional detailed explanation

- Active voice: "Add feature" not "Added feature"
- Why good messages matter: Future you, collaborators, debugging
- One logical change per commit
- Staging intentionally: Not just `git add .`
- Reviewing what you're committing: `git diff --staged`
- Commit frequency: Regular checkpoints, not huge dumps
- Relating commits to features/issues

Transitions: You've been creating commits throughout the course; now refine your technique. Professional commit practices make your Git history a valuable project resource.

Key Principles:

- Commit messages are documentation for future readers
- One logical change per commit enables easier debugging
- Descriptive messages save time later
- Clean commits demonstrate professionalism

Examples:

- Good: "Add user authentication with JWT tokens"
- Bad: "Update", "Fix stuff", "WIP"
- Good: "Fix header alignment on mobile devices"
- Bad: "Changes", "asldjf", "test"
- Commit series: Each adds one clear feature

04.3 Common Git Anti-Patterns to Avoid 📖 (~550 words)

Learning Objectives:

- Recognize common Git mistakes
- Understand consequences of each anti-pattern
- Apply correct approaches instead
- Develop healthy Git habits

Content Outline:

- Anti-pattern 1: Always using `git add .`
  - Problem: Committing unintended changes
  - Solution: Stage files intentionally
- Anti-pattern 2: Working directly on main branch
  - Problem: No safety net, breaks collaboration
  - Solution: Always use feature branches
- Anti-pattern 3: Poor commit messages
  - Problem: Unclear history, harder debugging
  - Solution: Descriptive, action-oriented messages
- Anti-pattern 4: Huge commits with many changes
  - Problem: Hard to review, hard to revert
  - Solution: Commit logical units frequently
- Anti-pattern 5: Using `git push --force` carelessly
  - Problem: Overwrites others' work, breaks collaboration
  - Solution: Only force on personal branches, understand implications
- Anti-pattern 6: Committing configuration files (.env, passwords)
  - Problem: Security risk, sensitive data exposed
  - Solution: Use .gitignore, keep configs out of repo
- Anti-pattern 7: Large uncommented commits
  - Problem: No context, unclear intent

- Solution: Explain why, not just what

Transitions: Throughout this course you've learned correct Git practices; now see the common mistakes to avoid. Recognizing these anti-patterns helps you develop clean, professional Git habits.

Key Principles:

- Anti-patterns are common for a reason—they're tempting shortcuts
- Each anti-pattern has real consequences for you or your team
- Professional developers avoid these mistakes consistently
- Good Git hygiene becomes automatic with awareness and practice

Examples:

- Real consequence: Force push overwrote teammate's work
- Security incident: Committed passwords to public repo
- Debugging nightmare: Massive commit changed 50 files
- Workflow chaos: Team working directly on main branch

Section 05: Assessment and Practice

05.0 Git Workflow Challenge 💻 (~650 words)

Learning Objectives:

- Apply complete Git workflow independently
- Navigate a realistic development scenario
- Make decisions about branching and merging
- Demonstrate Git proficiency through practice

Content Outline:

- Challenge scenario: Building a multi-feature project
- Setup: Initialize repository, create initial structure
- Task 1: Create and complete first feature on branch
- Task 2: Create second feature in parallel
- Task 3: Merge both features appropriately
- Task 4: Handle a bug fix scenario
- Task 5: Clean up branches
- Requirements: Proper naming, clean commits, logical structure
- Evaluation: Branch structure, commit quality, workflow execution

Transitions: You've learned Git piece by piece; now prove your skills by executing a complete workflow independently. This challenge simulates real development work.

Key Principles:

- Real development involves managing multiple features
- Clean workflows require planning and discipline
- Professional standards matter in every commit
- Git skills enable confident, organized development

Examples:

- Challenge setup with clear requirements
- Sample feature specifications to implement
- Expected branch structure at completion
- Quality criteria for evaluation

Hands-On Exercise: Complete a multi-stage Git workflow challenge:

1. Initialize a new project repository with basic structure
2. Create a development branch from main
3. Create feature/add-header branch and implement a header component
4. Create feature/add-footer branch (from dev) and implement footer
5. Merge both features to dev with proper commit messages
6. Create bugfix/footer-styling to fix an issue
7. Merge bugfix to dev, then merge dev to main
8. Clean up all feature and bugfix branches
9. Verify clean repository state with proper Git log history

Success Criteria:

- All features completed on appropriate branches
- Merge strategy followed correctly
- Commit messages clear and descriptive
- Clean final repository state
- Log shows logical, organized history
- Student can explain every step taken

05.1 Course Assessment 🧠 (~650 words)

Learning Objectives:

- Demonstrate understanding of Git concepts
- Apply Git knowledge to realistic scenarios
- Identify correct approaches to common situations
- Show proficiency with Git workflows

Assessment Format: Scenario-based questions

Question 1: Understanding Git States You've made changes to three files: index.html, style.css, and script.js. You run `git status` and see all three listed as modified. You want to commit only the HTML and CSS changes, saving the JavaScript changes for a separate commit.

What is the correct sequence of commands?

A) `git add index.html style.css` → `git commit -m "Update HTML and CSS"` → later: `git add script.js` → `git commit -m "Update JavaScript"`

B) `git commit -m "Update HTML and CSS"` → `git add index.html style.css` → later: `git add script.js` → `git commit -m "Update JavaScript"`

C) `git add .` → `git commit -m "Update HTML and CSS"` → `git remove script.js` → later: `git add script.js` → `git commit -m "Update JavaScript"`

D) `git stash script.js` → `git add index.html style.css` → `git commit -m "Update HTML and CSS"` → `git stash pop`

Question 2: Branch Management You're working on a new user authentication feature. Your main branch is stable and deployed to production. You've been asked to develop the authentication feature without affecting the main branch until it's complete and tested.

What is the correct Git workflow?

A) Work directly on main branch, test thoroughly before pushing B) Create feature/user-authentication branch from main, develop there, merge back when complete C) Create a personal branch named your-name-auth, develop and push directly to production D) Make all changes locally without committing until feature is complete, then commit everything at once

Question 3: Merge vs Rebase You've been working on feature/new-dashboard for several days. During this time, your team has merged several updates to the main branch. You want to incorporate those updates into your feature branch before finishing your work. You've already pushed your feature branch to the remote repository and a teammate is reviewing your code.

What should you do?

A) Run `git rebase main` on your feature branch to apply updates B) Run `git merge main` into your feature branch to incorporate updates C) Delete your feature branch and start over from updated main D) Continue working without incorporating updates until your feature is complete

Question 4: Fixing Mistakes You've just committed changes with message "Update header" but immediately realized you forgot to include one important file (navigation.js). You haven't pushed this commit yet.

What's the best way to include the forgotten file in your last commit?

A) Create a new commit with message "Add forgotten file" B) Use `git add navigation.js` → `git commit --amend` to add the file to previous commit C) Run `git reset HEAD~1`, then commit everything together D) Use `git revert HEAD`, then create a new commit with all changes

Question 5: Understanding Remote Operations You're working on a team project. You've made several commits on your local main branch. When you try to push with `git push origin main`, you receive an error saying the remote has work you don't have locally.

What does this mean and what should you do?

A) Your commits conflict with remote; delete your local work and pull fresh copy B) Someone else pushed commits to remote main; you should pull first, then push C) The remote repository is corrupted; contact your team lead D) You need to force push to overwrite the remote: `git push origin main --force`

Question 6: Repository Management You're starting a new project and want to version control it with Git. The project directory already exists with some initial files. You want to track these files and connect to a remote repository.

What is the correct sequence of steps?

A) `git remote add origin <url>` → `git push` B) `git init` → `git add .` → `git commit -m "Initial commit"` → `git remote add origin <url>` → `git push origin main` C) `git add .` → `git init` → `git push` D) `git commit -m "Initial commit"` → `git init` → `git push origin main`

Question 7: Git Workflow Strategy Your team follows a simplified gitflow with main, dev, and feature branches. You've completed a feature called "user-profile" and it's been tested on your feature branch. Now you need to integrate it into the project.

What is the correct workflow following simplified gitflow?

A) Merge feature/user-profile directly to main B) Merge feature/user-profile to dev, test, then merge dev to main C) Rebase feature/user-profile onto main, force push D) Create a new branch from main, cherry-pick your commits

Evaluation Criteria:

- 7/7 correct: Excellent Git understanding
- 5-6 correct: Good understanding, review specific topics
- 3-4 correct: Basic understanding, more practice needed
- 0-2 correct: Review course material and practice workflows

Section 06: Conclusion

06.0 Your Git Foundation is Built (~450 words)

Content Outline:

You've completed your journey into Git fundamentals! Let's recap what you've accomplished:

What You've Mastered:

You started by understanding how Git thinks—the three-state system, snapshots, and commits. You learned the essential commands that form the foundation of all Git work: status, add, commit, push, and pull. These core operations are tools you'll use every single day as a developer.

You then progressed to branching, understanding how Git enables parallel development through lightweight branches. You practiced creating feature branches, switching between them, and managing multiple lines of development—a workflow that keeps projects organized and developers sane.

The advanced operations section equipped you with tools for managing mistakes and complex scenarios: stashing work when context-switching, reverting commits safely, understanding merge versus rebase, and reading your project history through Git log. These skills transform you from someone who uses Git to someone who understands Git.

Finally, you learned professional workflows and best practices: simplified gitflow, quality commit practices, and common anti-patterns to avoid. These organizational patterns separate hobby projects from professional development.

Your Git Workflow:

At this point, you can confidently:

- Initialize repositories and track changes
- Create feature branches for new work
- Commit changes with clear, descriptive messages
- Navigate your project's history
- Merge completed features
- Collaborate through remote repositories
- Recover from common mistakes
- Follow professional Git workflows

Connection to the Bigger Picture:

Git is the foundation for collaborative development. With your Git skills solid, you're ready to explore GitHub in the next course—pull requests, issues, collaborative workflows, and the social aspects of modern software development.

These version control skills integrate perfectly with AI-assisted development. Understanding Git enables you to confidently iterate, experiment, and use AI tools effectively. You can try new approaches on branches, revert when experiments fail, and maintain clean project history even when working rapidly with AI assistance.

Next Steps:

The next course introduces GitHub, where your Git knowledge enables powerful collaboration. You'll learn pull requests, code review, project management through issues, and team workflows that leverage everything you've learned here.

Keep Practicing:

Git mastery comes through practice. Use Git for every project, even small ones. Create feature branches for every new feature. Write clear commit messages. Your Git skills compound with practice—each project makes you more confident and efficient.

Your version control foundation is solid. Everything you build from here benefits from clean Git practices and confident workflows. Well done!

End of Course Outline: Git for Developers